Contents

Teaching a Robot to Solve a Rubik's Cube with AI 2
- Table of Contents . . . 3
- The Challenge: Why the Rubik's Cube is Hard for AI . . . 3
  - The Scale of the Problem . . . 3
  - God's Number . . . 3
  - Why Standard AI Struggles . . . 3
  - The 12 Moves . . . 4
- Our Project: What We Built . . . 4
  - The Big Picture . . . 4
  - The Virtual Cube . . . 5
  - Talking to the Neural Network . . . 5
  - Project Files . . . 5
  - Running the AI . . . 6
- Background: How Machines Learn . . . 6
  - Two Ways to Teach a Machine . . . 6
  - Neural Networks: What Are They, Really? . . . 7
  - Why We Need a GPU . . . 8
  - Key Terms Quick Reference . . . 9
- Approach 1: Standard Reinforcement Learning (PPO/DQN) . . . 9
  - The Idea . . . 9
  - Two Algorithms We Tried . . . 10
  - What the Code Looks Like . . . 10
  - Why It Fails for the Rubik's Cube . . . 11
  - Reward Shaping: Trying to Help . . . 11
  - Our Results . . . 11
- Approach 2: Curriculum Learning . . . 11
  - The Idea . . . 12
  - How It Works . . . 12
  - Why It Eventually Fails: The Transfer Gap . . . 12
  - Catastrophic Forgetting . . . 13
  - Our Results . . . 13
- Approach 3: Hindsight Experience Replay (HER) . . . 13
  - The Idea . . . 13
  - How It Works . . . 14
  - Why It Doesn't Fit the Rubik's Cube . . . 14
  - Where HER Shines . . . 14
- Approach 4: Autodidactic Iteration (ADI) . . . 15
  - The Idea: A Completely Different Philosophy . . . 15
  - The Core Insight: Working Backwards . . . 15
  - A Small Catch: The Labels Aren't Perfect . . . 16
  - What the Network Learns: Two Jobs, One Brain . . . 16
  - The Network Architecture: Our v1/v2 Design . . . 16
  - Scaling Up: Making the Network Smarter (v3) . . . 17
  - Our v3 Architecture . . . 18
  - How the Trained Network Solves Cubes: Beam Search . . . 19
  - Why ADI Works So Well . . . 19

Our Results . . . 19
Approach 5: Imitation Learning . . . 20
The Idea . . . 20
How It Works . . . 20
The Compounding Error Problem . . . 21
Why We Didn't Use It . . . 21
Approach 6: Hybrid Methods . . . 21
The Idea . . . 21
Sim-to-Real Transfer (Our Next Step) . . . 21
Other Interesting Combinations . . . 22
Comparison: How the Approaches Stack Up . . . 22
Which Should You Choose? . . . 23
Our Experimental Results . . . 23
The Story of Five Training Runs . . . 23
Run 1: PPO with Basic Curriculum (January 31, 2026) . . . 23
Run 2: PPO with Improved Curriculum (February 2, 2026) . . . 23
Run 3: ADI v1 (February 3, 2026) . . . 24
Run 4: ADI v2 (February 6, 2026) . . . 24
Run 5: ADI v3 (February 7, 2026) — In Progress . . . 25
The Big Picture: How Each Run Compared . . . 25
What We'd Recommend . . . 25
Further Reading . . . 26
Glossary . . . 26
Our Hardware . . . 28

Teaching a Robot to Solve a Rubik's Cube with AI

> The short version: We tried three different AI approaches to teach a LEGO robot to solve a Rubik's Cube. The first two failed. The third — called Autodidactic Iteration — cracked it, achieving a 100% solve rate in just 20 minutes of training. This guide tells the full story.

You've probably solved a Rubik's Cube before — or at least tried. Maybe you followed a YouTube tutorial, memorised a few algorithms, and eventually got all the colours to line up. But what if you had to teach a computer to do it, without giving it any algorithms at all? What if the computer had to figure it out entirely on its own?

That's what this project is about. We built an AI system that learns to solve the Rubik's Cube through experience, then connected it to a physical LEGO robot that can pick up a real cube and solve it. Along the way, we tried several different approaches, failed spectacularly at some, and eventually found one that works remarkably well.

This guide explains everything we did — the ideas behind each approach, why some failed, and how we eventually succeeded. You don't need a computer science degree to follow along. If you're comfortable with basic maths (GCSE level) and have a rough idea of what a computer program is, you'll be fine. We'll explain every technical concept as we go.

Table of Contents

1. The Challenge: Why the Rubik's Cube is Hard for AI
2. Our Project: What We Built
3. Background: How Machines Learn
4. Approach 1: Standard Reinforcement Learning (PPO/DQN)
5. Approach 2: Curriculum Learning
6. Approach 3: Hindsight Experience Replay (HER)
7. Approach 4: Autodidactic Iteration (ADI) — our chosen approach
8. Approach 5: Imitation Learning
9. Approach 6: Hybrid Methods
10. Comparison: How the Approaches Stack Up
11. Our Experimental Results
12. What We'd Recommend
13. Further Reading
14. Glossary

The Challenge: Why the Rubik's Cube is Hard for AI

The Scale of the Problem

The Rubik's Cube has 43,252,003,274,489,856,000 possible configurations — that's 43 quintillion. To put that in perspective: if you could check one billion configurations per second, it would take you over 1.3 billion years to check them all. That's roughly a quarter of the age of the Earth.

Out of all those configurations, there is exactly one solved state. Finding it is like finding a specific grain of sand on every beach on Earth — combined.

God's Number

In 2010, researchers proved that any Rubik's Cube can be solved in 20 moves or fewer. This number is called "God's Number" because only an all-knowing being could always find the shortest solution. The average optimal solution is about 18 moves.

For our robot, we don't need perfection. A solution in 25-30 moves is fine — even human speed-cubers typically use 50-60 moves.

Why Standard AI Struggles

There are five specific reasons why the Rubik's Cube is exceptionally hard for AI:

1. Almost no feedback (the "sparse reward" problem)

Imagine learning to play a song on piano, but the only feedback you ever get is either "that was the exact right song" or silence. No hints about which notes were right, no partial credit. That's what the AI faces: out of 43 quintillion states, only one gives a positive signal. Making random moves and hoping to stumble on the solution is like winning the lottery — except the odds are billions of times worse.

2. Looks can be deceiving

A cube that's one move from solved and a cube that's twenty moves from solved can look almost identical — both have most stickers in the right place. Counting correct stickers is deeply misleading: you can have 45 out of 54 stickers correct and still be 15 moves from solved. The AI can't trust simple measures of "how close am I?"

3. Too many possibilities

At each step, the AI can choose from 12 moves. After 10 moves, there are about 62 billion possible sequences. After 20 moves, that number exceeds a sextillion (10^{21}). You can't try them all — the AI has to learn which moves are worth trying.

4. Every move matters

A wrong move at step 3 might not become obvious until step 15. Each decision depends on all the previous ones. This is much harder than problems where each decision is independent.

5. The cube looks different from every angle

The same cube state can appear different depending on how you hold it. A naive AI might learn the same pattern six times — once for each way the cube could be oriented — wasting capacity.

The 12 Moves

A standard Rubik's Cube has 6 faces, each of which can be turned clockwise or counter-clockwise:

Face	Moves	What It Means
Front	F / F'	Turn the front face clockwise / counter-clockwise
Back	B / B'	Turn the back face
Left	L / L'	Turn the left face
Right	R / R'	Turn the right face
Up	U / U'	Turn the top face
Down	D / D'	Turn the bottom face

That gives us 12 possible actions at each step. The prime symbol (') means counter-clockwise — so F is "turn the front face clockwise" and F' is "turn it back."

Our Project: What We Built

Before diving into the different approaches we tried, here's a quick overview of what we actually built. Don't worry about understanding every detail here — we'll explain everything properly in the sections that follow.

The Big Picture

Our project has three main parts:

1. A Rubik's Cube simulator — a virtual cube inside the computer that we can scramble and solve millions of times per hour, much faster than a physical cube

2. An AI brain — a neural network (we'll explain what this is shortly) that learns to look at a scrambled cube and figure out what moves to make
3. A robot bridge — code that sends the AI's solution to a physical LEGO robot, which then turns the real cube

The Virtual Cube

The first thing we needed was a perfect simulation of a Rubik's Cube inside the computer. Think of it like a video game version of the cube — it follows all the same rules as a real cube, but we can manipulate it instantly with code.

Our simulator stores the cube as a grid of numbers. Each face has 9 sticker positions, and each sticker has a colour represented as a number (0-5). In the solved state, all stickers on face 0 are colour 0, all stickers on face 1 are colour 1, and so on.

```
# The cube is a 6x9 grid: 6 faces, 9 stickers each
# In the solved state:
# Face 0 (Front):  [0, 0, 0, 0, 0, 0, 0, 0, 0]  (all colour 0)
# Face 1 (Back):   [1, 1, 1, 1, 1, 1, 1, 1, 1]  (all colour 1)
# Face 2 (Left):   [2, 2, 2, 2, 2, 2, 2, 2, 2]  (all colour 2)
# ...and so on for all 6 faces
```

To check if the cube is solved, we just check whether every face still has all the same colour — a very fast operation.

When we "make a move," the code does two things: it rotates the 9 stickers on the face being turned (like spinning a tic-tac-toe grid), and it shuffles the stickers on the four neighbouring faces that get dragged along. Getting these shuffles exactly right was one of the trickiest parts of the project — one wrong sticker position and the whole simulation would be subtly broken.

Talking to the Neural Network

A neural network can't look at our grid of numbers directly — it needs the data in a specific format. We convert each sticker into a set of 6 numbers where exactly one is "1" and the rest are "0." This is called one-hot encoding, and we'll explain why it matters in the Background section.

The key point: our cube becomes a list of 324 numbers (54 stickers × 6 possible colours) that we feed into the neural network.

Project Files

```
mindcuber-rl/
├── src/
│   ├── cube_env.py       # The virtual cube simulator
│   ├── adi_train.py    # Our main AI training code (the approach that worked)
│   ├── adi_solve.py      # Uses the trained AI to solve cubes
│   ├── train.py          # Our first AI approach (PPO — didn't work well)
│   └── spike_bridge.py   # Sends solutions to the LEGO robot
├── models/               # Saved AI "brains" after training
└── requirements.txt      # Software dependencies
```

Running the AI

Once trained, we can use our AI to solve cubes from the command line:

```
# Test the AI on 100 random cubes at different difficulty levels
python src/adi_solve.py models/adi_v2/best_model.pt --mode evaluate

# Example output:
# Depth  5: 100.0% solved | Avg moves:   4.9 | Avg time: 0.004s
# Depth 10:  99.0% solved | Avg moves:  10.5 | Avg time: 0.041s
# Depth 15:  42.0% solved | Avg moves:  18.4 | Avg time: 0.312s
# Depth 20:  25.0% solved | Avg moves:  22.1 | Avg time: 0.587s
```

We'll show more of the actual code later, in context, when we explain each approach. For now, the important thing is that we have a working cube simulator and a trained AI that can solve cubes in under a second.

Background: How Machines Learn

Before we look at the specific approaches we tried, you need to understand a few core ideas. This section is the foundation for everything that follows, so take your time with it.

Two Ways to Teach a Machine

There are two fundamentally different ways to teach a computer to do something. Understanding the difference is crucial because our project tried both — and the choice of method turned out to be the single most important decision we made.

Way 1: Supervised Learning — "Here's the Answer Key" Imagine you're revising for a maths exam. Your teacher gives you 1,000 practice questions, each with the correct answer. You work through them, check your answers, and gradually learn the patterns. After enough practice, you can solve questions you've never seen before.

That's supervised learning. You give the computer millions of examples where you already know the right answer, and it learns to predict the answer for new examples it hasn't seen.

```
Example training data for our cube:
  "Here's a scrambled cube" → "The best move is F"
  "Here's another scrambled cube" → "The best move is R'"
  "Here's yet another" → "The best move is U"
  ... millions more examples ...
```

The catch? You need to already know the correct answers. For the Rubik's Cube, that means you'd need an existing solver to generate the training data — which is a bit of a chicken-and-egg problem. (Spoiler: our winning approach, ADI, finds a clever way around this.)

Way 2: Reinforcement Learning — "Figure It Out Yourself" Now imagine learning to ride a bicycle. Nobody gives you a textbook of correct muscle movements. Instead, you just get on the bike, try

things, fall off, try again, and gradually figure out what works. Your only feedback is whether you're staying upright (good) or hitting the ground (bad).

That's reinforcement learning (RL). The computer interacts with the cube — making moves, seeing what happens — and the only feedback it gets is a score: +100 points for solving the cube, -1 point for each move it makes (to encourage efficiency). Over thousands of attempts, it's supposed to learn which moves lead to good outcomes.

The key components are:

- Agent: The AI making decisions (our neural network)
- Environment: The thing it's interacting with (our cube simulator)
- State: What the AI can see right now (the current cube configuration)
- Action: What the AI can do (one of 12 face rotations)
- Reward: The score it gets (+100 for solving, -1 per move)
- Policy: The AI's current strategy — its "instinct" for what to do in any given situation

Neural Networks: What Are They, Really?

You've probably heard the term "neural network" — it sounds very sci-fi, but the reality is surprisingly simple. A neural network is just a mathematical function that takes some numbers as input and produces some numbers as output. What makes it special is that it can learn to produce useful outputs by adjusting millions of internal settings.

The Basics: Layers and Weights Picture a neural network as a series of layers, like floors in a building. Data enters at the ground floor, gets transformed at each floor, and comes out the top as a prediction.

```
Ground floor:  324 numbers go in (our cube state)
                    ↓
Floor 1:      4096 "neurons" each look at the input and detect simple patterns
                    ↓
Floor 2:       2048 neurons combine those patterns into more complex ones
                    ↓
Floor 3:       1024 neurons build even higher-level understanding
                    ↓
Floor 4:       512 neurons create a final summary
                    ↓
Top floor:    The prediction comes out (e.g., "this cube is 7 moves from solved")
```

Each neuron does something very simple: it takes all the numbers from the previous layer, multiplies each one by a weight (just a number), adds them all up, and passes the result to the next layer. That's it. The magic is that when you have millions of these simple operations chained together, the network can learn incredibly complex patterns.

Analogy: Think of each neuron as a person in a crowd doing a simple task — like holding up a green flag if they hear a loud noise, or a red flag otherwise. Individually, each person is doing something trivial. But if you arrange thousands of people in layers, where each person watches the flags of the people in front of them, the crowd as a whole can do something remarkably sophisticated — like recognising a face or understanding a sentence.

The weights are the key. They determine what each neuron pays attention to. A neuron with a high weight on one input and a low weight on another is essentially saying "I care a lot about this piece of information, but not that one." When we "train" a neural network, we're adjusting these millions of weights until the network produces useful predictions.

How Training Works: The Learning Loop Training a neural network is like tuning a radio. You start with static (random weights = random predictions), and you gradually adjust the dial until you get a clear signal (accurate predictions). Here's how:

1. Show the network an example — feed in a scrambled cube state
2. Get its prediction — the network guesses "this cube is 12 moves from solved"
3. Check the answer — we know it's actually 7 moves from solved
4. Measure the error — the network was off by 5. We square this to get the loss: $5^2 = 25$. (Squaring makes big errors count much more than small ones, which is what we want.)
5. Figure out which weights caused the error — this is called backpropagation. It works backwards through the network, calculating how much each weight contributed to the mistake. Think of it like tracing a wrong answer on an exam back to the specific misconception that caused it.
6. Nudge the weights to reduce the error — this is called gradient descent. Each weight gets adjusted slightly in the direction that would have made the prediction more accurate. The learning rate controls how big these nudges are — too big and the network overshoots; too small and learning takes forever.
7. Repeat millions of times — each time, the predictions get a little bit better.

Analogy: Imagine you're blindfolded on a hilly landscape, trying to find the lowest valley. You can't see, but you can feel the slope under your feet. At each step, you walk a little bit downhill. That's gradient descent — you're always moving in the direction that reduces your "height" (the error). Eventually, you reach a valley (low error = good predictions).

Batches and Epochs We don't train on one example at a time — that would be too noisy (one unusual example could send the weights in the wrong direction). Instead, we train on batches of examples at once. Our batch size is 1,024, meaning we show the network 1,024 scrambled cubes at the same time, average out the errors, and then update the weights. This gives much smoother, more reliable learning.

An epoch is one complete pass through all the training data. If we have 20 million training examples and a batch size of 1,024, one epoch is about 19,500 batches. We typically train for 100 epochs, meaning the network sees each example 100 times. Early epochs produce big improvements; later epochs fine-tune the details.

Why We Need a GPU

All those multiplications — millions of weights times thousands of inputs, repeated billions of times — add up to an enormous amount of arithmetic. A normal computer processor (CPU) does calculations one at a time, like a single very fast mathematician. A GPU (Graphics Processing Unit) is more like a stadium full of slower mathematicians who all work simultaneously.

GPUs were originally designed for video games — rendering 3D graphics requires doing the same calculation for millions of pixels at once. It turns out that training neural networks requires exactly

the same kind of parallel computation. Our NVIDIA DGX Spark GPU can do these calculations 10-100× faster than a CPU, turning what would be days of training into hours.

Key Terms Quick Reference

Don't worry about memorising these — they'll be explained in context as we go. This is just a handy reference if you forget what something means.

Term	Plain English
Epoch	One complete pass through all the training data
Batch	A group of examples processed together (ours: 1,024 at a time)
Loss	A number measuring how wrong the network is (lower = better)
Learning rate	How big the weight adjustments are at each step
Overfitting	When the network memorises answers instead of learning patterns (like cramming for an exam vs. actually understanding the material)
Inference	Using a trained network to make predictions (the "exam" after all the "revision")
Policy	The AI's strategy — its rule for deciding what to do in any situation
Value function	The AI's estimate of how close a cube state is to being solved

---

Approach 1: Standard Reinforcement Learning (PPO/DQN)

> We tried this — our first attempt. It plateaued at difficulty 7 with a 35% solve rate. See Run 1.

The Idea

This is the most intuitive approach: let the AI interact with the cube, give it points when it solves it, and let it figure out the rest through trial and error. This is the "learning to ride a bicycle" approach from the Background section.

The setup is simple:

- The AI sees the current cube state (324 numbers)
- It picks one of 12 possible moves
- It gets a score: +100 for solving the cube, -1 for each move (to encourage short solutions)
- It keeps going until it either solves the cube or runs out of moves (we allowed 50)

One important property of the Rubik's Cube makes this setup valid: the AI only needs to see the current state to make a good decision. It doesn't need to remember what moves it made before — the cube's current configuration contains all the information needed. (In technical terms, this makes it a "Markov Decision Process," but you don't need to remember that name.)

Two Algorithms We Tried

We experimented with two well-known RL algorithms. Both are available in a Python library called Stable-Baselines3, which handles all the complicated maths for you.

PPO (Proximal Policy Optimization) PPO, developed by OpenAI in 2017, learns a policy — a strategy that says "when the cube looks like this, I should probably do that." It works by:

1. Playing thousands of games (scramble the cube, try to solve it)
2. Looking at which moves led to good outcomes and which didn't
3. Making good moves slightly more likely and bad moves slightly less likely
4. Repeating

The clever part of PPO is that it limits how much the strategy can change in each update. Without this limit, the AI might make a big change that accidentally ruins everything it's learned so far — like a student who, after getting one question wrong, decides to completely change their approach and forgets everything they knew. PPO prevents this by saying "you can adjust your strategy, but only a little bit at a time."

PPO throws away its old experiences after each update and collects fresh ones. This is like a student who only revises from today's lesson, never looking back at old notes. It's stable but wasteful.

DQN (Deep Q-Network) DQN, the algorithm that famously learned to play Atari games at super-human level (DeepMind, 2015), takes a different approach. Instead of learning "what should I do?", it learns "how good is each possible move?"

For every cube state, DQN estimates a score for each of the 12 possible moves. The score represents "if I make this move and then play perfectly from there, how much total reward will I get?" The AI then simply picks the move with the highest score.

DQN keeps a memory bank of past experiences and replays them during training — like a student who keeps all their old homework and occasionally revises from it. This is more efficient than PPO's approach of throwing everything away.

What the Code Looks Like

Thanks to the Stable-Baselines3 library, the actual code is surprisingly short:

```
from stable_baselines3 import PPO

# Create the cube environment and the AI
env = RubiksCubeEnv(scramble_moves=5, max_steps=50)
model = PPO('MlpPolicy', env, device='cuda')  # 'cuda' = use the GPU

# Train for 10 million steps
model.learn(total_timesteps=10_000_000)
```

Behind the scenes, this creates a neural network, runs millions of episodes, and gradually improves the AI's strategy. The library handles all the gradient descent, backpropagation, and weight updates we discussed in the Background section.

Why It Fails for the Rubik's Cube

The fundamental problem is that the AI almost never gets positive feedback. Here's what happens:

1. The AI starts with a completely random strategy (equal chance of picking any of the 12 moves)
2. It scrambles the cube and starts making random moves
3. The chance of randomly solving even a 5-move scramble in 50 random moves is astronomically small
4. Without ever getting the +100 reward, the AI has no signal to learn from
5. All it ever experiences is the -1 penalty for each move, which tells it nothing about how to solve the cube

Analogy: Imagine trying to learn to play a song on piano, but the only feedback you ever get is "that wasn't the right song." No hints about which notes were correct, no partial credit, nothing. You'd just be pressing random keys forever. That's what standard RL faces with the Rubik's Cube.

Reward Shaping: Trying to Help

We tried giving the AI partial credit by counting how many stickers are in the correct position after each move. If it moves a sticker to the right place, it gets a small bonus; if it moves one away, a small penalty.

This helps a bit, but it's deeply misleading. As we discussed in the Challenge section, you can have 45 out of 54 stickers correct and still be 15 moves from solved. The AI learns to chase high sticker counts without actually making progress — like a student who learns to write essays that look good but don't actually answer the question.

Our Results

Training Run	Steps	Network Size	Max Difficulty	Solve Rate
Run 1	10M	512→256→128	7 moves	35%

The AI learned to solve 1-6 move scrambles reliably but hit a wall at difficulty 7. Beyond that depth, random exploration almost never finds the solution, so the AI has nothing to learn from.

Verdict: Standard RL alone doesn't work for the Rubik's Cube. The sparse reward problem is simply too severe. But this attempt wasn't wasted — it taught us why the problem is hard and motivated our next approach.

Approach 2: Curriculum Learning

> We tried this — our second attempt. Improved to 54% at difficulty 7 and 9% on 10-move scrambles, but still hit the same ceiling. See Run 2.

The Idea

Curriculum learning borrows from how humans learn: you don't start a child with calculus — you start with counting, then addition, then multiplication, and gradually build up.

Applied to the Rubik's Cube, this means: don't throw a fully scrambled cube at the AI from day one. Instead, start with a cube that's only been scrambled by 1 move. A 1-move scramble can be solved by just picking the right move out of 12 — even a random guess will get it right about 8% of the time. That's enough for the AI to start learning.

Once the AI can reliably solve 1-move scrambles, increase the difficulty to 2 moves. Then 3, then 4, and so on. Each level builds on the skills learned at the previous level.

How It Works

```
Level 1: Scramble with 1 move
  → AI learns: "When one face is wrong, undo it"
  → Solving 85% of the time? Move to level 2.

Level 2: Scramble with 2 moves
  → AI learns: "When two faces are wrong, try these sequences"
  → Solving 85%? Move to level 3.

Level 3: Scramble with 3 moves
  → AI builds on what it learned at levels 1 and 2
  → Solving 85%? Move to level 4.

...and so on, up to level 20 (God's Number)
```

The key question is: when should the AI move up a level? We tried two approaches:

Approach 1 (Run 1): Move up after 10 consecutive solves. This turned out to be unreliable — the AI could get lucky 10 times in a row without truly mastering the level.

Approach 2 (Run 2): Move up when the AI solves at least 85% of cubes over a window of 50 attempts. This is much more robust — 85% over 50 tries is hard to fluke.

Why It Eventually Fails: The Transfer Gap

Curriculum learning works beautifully for the first few levels. But around level 7, something frustrating happens: the AI gets stuck. No matter how long it trains, it can't break past this barrier.

The reason is what we call the transfer gap. The skills the AI learns for easy scrambles don't fully transfer to harder ones. Here's why:

When a cube is scrambled by 1-3 moves, it still looks "almost solved" — most stickers are in the right place, and the AI learns to spot and fix small local problems. But a cube scrambled by 7+ moves looks completely different. The patterns are global, not local. It's like the difference between fixing a few typos in an essay versus rewriting the whole thing — fundamentally different skills.

Analogy: Imagine learning to swim in a paddling pool. You master the basics — kicking, arm movements, breathing. Then someone drops you in the ocean. The waves, the currents, the depth

— everything is different. Your paddling pool skills help a bit, but they're not enough. That's the transfer gap.

Catastrophic Forgetting

There's another problem: as the AI trains on harder scrambles, it can gradually forget how to solve easier ones. The neural network has limited capacity (a fixed number of weights), and adjusting those weights for level-7 problems can overwrite the patterns that were tuned for level-3 problems.

This is called catastrophic forgetting, and it's a fundamental challenge in AI. It's like studying so hard for your physics exam that you forget everything you learned for chemistry.

Our Results

Run	Steps	Advancement Rule	Network Size	Max Level	Solve Rate	10-Move Test
Run 1	10M	10 consecutive solves	512→256→128	7	35%	0%
Run 2	30M	85% over 50 attempts	1024→512→256	6	54%	9%

Both runs hit the same wall at level 7, despite Run 2 using a 3× larger network and 3× more training. This told us something important: the problem isn't that we need a bigger network or more training time. The problem is fundamental to the RL + curriculum approach itself.

The 9% solve rate on 10-move scrambles in Run 2 was interesting — it showed the AI had learned something about deeper solving, but not enough to be reliable.

Verdict: Curriculum learning is a big improvement over plain RL, and it's a natural first thing to try. But for the Rubik's Cube, it hits a ceiling around difficulty 7-8. To go further, we needed a completely different approach — which brings us to ADI.

Approach 3: Hindsight Experience Replay (HER)

> We skipped this — it's a clever idea, but not well-suited to the Rubik's Cube. Here's why we considered it and why we moved on.

The Idea

HER, introduced by OpenAI in 2017, is one of the most creative ideas in AI. It addresses the sparse reward problem by asking: "What if we pretended the AI was trying to do what it actually did?"

Analogy: A child is learning to throw a ball into a basket. They throw and miss — the ball lands on the floor three feet to the left. Normally, that's just a failure. But a clever teacher says: "Great throw! You hit exactly that spot on the floor!" The child didn't learn to hit the basket, but they learned

something about throwing — how arm angle and force affect where the ball goes. Over time, this knowledge transfers to hitting the actual target.

HER applies this to the Rubik's Cube. The AI tries to solve the cube and fails — it ends up in some random state that isn't solved. Instead of throwing away this experience as a failure, HER says: "Let's pretend you were trying to reach that random state. In that case, you succeeded!" This gives the AI positive feedback to learn from, even though it never solved the cube.

How It Works

```
The AI tries to solve the cube but fails:
  Goal:       Solved cube
  What happened: State A → State B → State C → State D (not solved)
  Feedback:   "You failed" (no learning signal)

HER creates bonus training examples:
  Pretend goal was State D:
    State A → State B → State C → State D (success! You reached State D!)
  Pretend goal was State B:
    State A → State B (success! You reached State B!)
```

Over many episodes, the AI learns to navigate between different cube states. The hope is that eventually, it becomes good enough at navigating that it can navigate to the solved state too.

Why It Doesn't Fit the Rubik's Cube

HER is brilliant for problems with many possible goals — like a robot arm that needs to reach different positions, or a character in a game that needs to navigate to different locations. But the Rubik's Cube has exactly one goal: the solved state.

Teaching the AI to navigate between random cube states is like teaching someone to drive to random addresses when they only ever need to get to one specific place. Most of what they learn is wasted effort.

There are other problems too: - The AI would need to handle 43 quintillion possible "pretend goals" — far too many - There's no natural notion of "getting closer" to an arbitrary cube state, so the AI can't build up skills gradually - The network needs to take both the current state and the goal state as input, doubling its complexity

Where HER Shines

HER is transformative for robotics — teaching robot arms to pick up objects, stack blocks, and open doors. In these tasks, learning to reach arbitrary positions is genuinely useful for eventually reaching the target position. The Rubik's Cube doesn't have this property.

Verdict: A clever technique, but wrong for our problem. We moved on to ADI, which is specifically designed for puzzles with a known goal state.

Approach 4: Autodidactic Iteration (ADI)

> This is our chosen approach — it achieved a 100% solve rate on 10-move scrambles after just 20 minutes of training. This is the one that worked.

The Idea: A Completely Different Philosophy

After two failed attempts with reinforcement learning, we tried something radically different. ADI (published by researchers at UC Irvine in 2019 under the name DeepCubeA) throws out the entire RL framework and asks a brilliantly simple question: "What if we don't use trial and error at all?"

Remember the core problem with RL: the AI makes random moves and almost never solves the cube, so it never gets positive feedback and can't learn. ADI sidesteps this entirely by switching from reinforcement learning to supervised learning — the "answer key" approach from the Background section.

But wait — supervised learning needs correct answers. Where do we get correct answers for the Rubik's Cube without already having a solver?

The Core Insight: Working Backwards

This is the cleverest part of the whole project. Instead of starting with a scrambled cube and trying to find the solution (hard), we start with the solved cube and scramble it ourselves (easy). Since we did the scrambling, we know exactly how far each state is from solved.

Analogy: Imagine you're lost in a maze. The RL approach drops you at a random spot and says "find the exit." You wander randomly, almost never finding it, and learn nothing.

ADI takes the opposite approach: start at the exit, walk one step into the maze, and write on the wall "1 step from exit." Walk another step: "2 steps from exit." Keep going: "20 steps from exit." Do this thousands of times along different paths, and you've created a detailed map of the maze — without ever having to actually solve it.

For the Rubik's Cube:

```
Start with solved cube (distance = 0)
Apply 1 random move  → record: "this state is 1 move from solved"
Apply 2 random moves → record: "this state is 2 moves from solved"
Apply 3 random moves → record: "this state is 3 moves from solved"
...
Apply 20 random moves → record: "this state is 20 moves from solved"
```

We also record which move was applied last — because the reverse of that move would take us one step closer to solved. So each training example contains three things:

1. The cube state (what the cube looks like)
2. The distance (how many moves from solved)
3. The best move (the reverse of the last scramble move)

We generate millions of these examples in minutes. No solving required. No trial and error. Just scramble, record, repeat.

A Small Catch: The Labels Aren't Perfect

There's a subtlety worth mentioning: applying 10 random moves doesn't guarantee the cube is exactly 10 moves from solved. Some moves might cancel each other out — for example, turning the front face clockwise and then counter-clockwise gets you back where you started (2 moves applied, 0 moves from solved).

In practice, this rarely matters. For scrambles of 5+ moves, the actual distance is usually very close to the number of moves applied. The labels are slightly imperfect, but the network learns to handle this — and the search process (explained below) compensates for any inaccuracy.

What the Network Learns: Two Jobs, One Brain

We train a single neural network to do two jobs simultaneously:

Job 1 — The Value Head: "How far away is this cube from being solved?"

This is a number-guessing task. The network looks at a scrambled cube and predicts a number — say, "7 moves from solved." During training, we know the actual answer (because we scrambled it ourselves), so we can measure how wrong the prediction is and adjust the weights.

The error is measured by squaring the difference: if the network guesses 12 but the answer is 7, the error is $(12-7)^2 = 25$. Squaring ensures that big mistakes are penalised much more than small ones.

Job 2 — The Policy Head: "Which move should I make next?"

This is a multiple-choice task. The network looks at a scrambled cube and assigns a probability to each of the 12 possible moves — "I'm 60% sure the best move is F, 20% sure it's R', 10% sure it's U…" During training, we know which move is correct (the reverse of the last scramble move), so we can push the network to assign higher probability to the right answer.

Why do both jobs in one network? Because both tasks require understanding the cube's structure. The shared "body" of the network learns a deep understanding of cube states, and then two small "heads" use that understanding for their specific tasks. This is more efficient than training two separate networks, and the two tasks actually help each other — understanding distance helps predict moves, and vice versa.

The Network Architecture: Our v1/v2 Design

Here's what our first network looks like. Don't worry about every detail — the key idea is that data flows through several layers, getting transformed at each step, before splitting into two outputs:

```
Input: 324 numbers (the cube state)
    ↓
Layer 1: 4096 neurons — detect basic patterns
    ↓
Layer 2: 2048 neurons — combine patterns
    ↓
Layer 3: 1024 neurons — build understanding
    ↓
Layer 4: 512 neurons — create summary
```

```
    ↓                          ↓
VALUE HEAD                 POLICY HEAD
"How far from solved?"     "Which move is best?"
(outputs 1 number)         (outputs 12 probabilities)
```

This network has about 12.6 million adjustable weights. That sounds like a lot, but it's tiny by modern AI standards — ChatGPT has over 100 billion.

Scaling Up: Making the Network Smarter (v3)

Our v1 and v2 networks solved 10-move scrambles perfectly, but struggled with 15-20 move scrambles. The network was running out of capacity — like trying to store a university textbook's worth of knowledge in a pocket notebook.

We improved the network in three ways for v3:

1. Wider Layers: More Patterns Per Level More neurons in each layer means the network can detect more patterns simultaneously. Think of it like describing a painting: with 100 words you can capture the broad strokes, but with 1,000 words you can describe the subtle details too. Deeply scrambled cubes have more subtle patterns to recognise.

2. Deeper Network: More Levels of Understanding More layers means the network can build more levels of abstraction:

```
Layer 1: "Which stickers are where?" (raw facts)
Layer 2: "Which pieces are correct?" (combining facts)
Layer 3: "How do groups of pieces relate?" (structural understanding)
Layer 4: "What's the overall state of the cube?" (global picture)
Layer 5+: Even more sophisticated understanding...
```

It's like the difference between a 3-step recipe and a 6-step recipe. Some dishes need more intermediate stages.

The problem with going deeper: When the network has many layers, the learning signal (the gradient) has to travel backwards through all of them during training. At each layer, the signal gets weaker — like a game of Chinese whispers where the message degrades with each retelling. After enough layers, the signal reaching the first layers is so faint that they stop learning entirely. This is called the vanishing gradient problem.

3. Residual Connections: The Key Innovation This is the most important upgrade, and it's worth understanding properly because it's one of the most influential ideas in all of AI.

The problem: Deeper networks should be better (more layers = more understanding), but in practice they were often worse because of the vanishing gradient problem.

The solution: Add shortcuts. Instead of forcing data to go through every layer in sequence, let it skip over layers:

```
Without skip connection:
  Data → [Layer A] → [Layer B] → Output
  Layer B must learn the ENTIRE transformation.
```

```
With skip connection:
  Data → [Layer A] → [Layer B] → Output
    └──────────────────────────────↗
  The original data is ADDED to Layer B's output.
  Layer B only needs to learn the DIFFERENCE (the "residual").
```

Why this helps — three reasons:

Reason 1: "Do nothing" is free. If a layer doesn't need to change the data, it doesn't have to learn anything — the skip connection passes the data through automatically. Without skip connections, the layer would have to learn to copy its input exactly, which is surprisingly hard.

Analogy: Imagine editing a document. Without skip connections, you'd have to rewrite the entire document from scratch at each revision. With skip connections, you keep the current document and just write down the changes. Much easier.

Reason 2: The gradient highway. The skip connections create a direct path for the learning signal to flow backwards through the network without getting weakened. Even if the signal through the layers themselves fades to nothing, the signal through the shortcuts stays strong. This solves the vanishing gradient problem.

Reason 3: Safety net. If a layer makes a mistake, the skip connection preserves the original data, limiting the damage. This makes the network more robust.

Residual connections were introduced in 2015 by Microsoft Research in a paper that has been cited over 200,000 times. They're now used in virtually every modern AI system, including the ones behind ChatGPT, image generators, and self-driving cars.

Our v3 Architecture

For v3, we combined all three improvements. Here's the structure:

```
Input (324 numbers)
    ↓
INPUT PROJECTION: Expand to 4096, then to 2048 neurons
    ↓
RESIDUAL BLOCK 1: Process at 2048 width, with skip connection
    ↓
RESIDUAL BLOCK 2: Same structure
    ↓
RESIDUAL BLOCK 3: Same structure
    ↓
RESIDUAL BLOCK 4: Same structure
    ↓
OUTPUT PROJECTION: Compress to 512 neurons
    ↓                          ↓
VALUE HEAD                 POLICY HEAD
(1 number)                 (12 probabilities)
```

This network has 44.7 million weights — 3.5× more than v1/v2. The four residual blocks give it the effective depth of about 10 layers, while the 2048-wide blocks give it plenty of capacity at each

level.

Did we design this ourselves? Yes. While we took inspiration from the original DeepCubeA paper, the specific architecture — how many blocks, how wide, the projection structure — is our own design, tuned for our hardware (the DGX Spark) and our training budget (20 million samples).

How the Trained Network Solves Cubes: Beam Search

Once trained, the network doesn't solve cubes on its own — it's not accurate enough for that. Instead, it guides a search algorithm called beam search. Think of it like this:

Greedy approach (doesn't work well): Ask the network "what's the best move?", make that move, ask again, repeat. This fails because one wrong prediction sends you down the wrong path with no way to recover.

Beam search (what we use): Instead of following one path, explore many paths simultaneously. At each step:

1. Take all your current candidates (starting with just the scrambled cube)
2. For each candidate, ask the network "what are the best few moves?"
3. Try each of those moves, creating new candidate states
4. Ask the network "how far is each new state from solved?"
5. Keep only the 100 most promising candidates (this is the "beam width")
6. If any candidate is solved, you're done!
7. Otherwise, go back to step 2

The beam width controls the trade-off between speed and reliability: - Beam width 1 = just follow the best path (fast but unreliable) - Beam width 100 = explore 100 paths at once (good balance) - Beam width 500 = explore 500 paths (slower but solves harder cubes)

A crucial optimisation: Instead of asking the network about each candidate one at a time, we send all 100+ candidates to the GPU in a single batch. This is 100-1000× faster than processing them individually, and it's the single most important performance trick in our code.

Why ADI Works So Well

Looking back at why RL failed, ADI solves every single problem:

1. No exploration problem — we generate training data by scrambling from solved, so we never need to "discover" the solution through random exploration
2. Every example has useful feedback — every training state has a distance label, not just the solved state
3. The training data matches what the solver sees — states generated by scrambling are exactly the kind of states the solver encounters
4. Search compensates for mistakes — even if the network makes a wrong prediction, beam search explores many paths, so one mistake doesn't ruin everything
5. It scales predictably — more data = better performance, bigger network = better performance, wider beam = better performance

Our Results

Version	Training Data	Network	Beam Width	5-move	10-move	15-move	20-move
v1	1M samples	12.6M params	100	100%	100%	19%	4%
v1	1M samples	12.6M params	200	100%	100%	50%	20%
v2	5M samples	12.6M params	100	100%	99%	42%	25%
v3	20M samples	44.7M params	100	TBD	TBD	TBD	TBD

The progression tells a clear story: - v1 → v2: 5× more training data doubled the 15-move rate (19% → 42%) and increased the 20-move rate from 4% to 25% - Wider beam search on v1 nearly tripled the 15-move rate (19% → 50%), showing that search compensates for network imperfections - v3 combines 20× more data, 3.5× more parameters, and residual connections — training is currently running on the DGX Spark

Verdict: ADI is the clear winner. It's the approach we chose for our robot, and it's the approach behind the state-of-the-art DeepCubeA system. The combination of self-generated training data, a dual-headed neural network, and beam search produces reliable, near-optimal solutions.

Approach 5: Imitation Learning

> We skipped this — it works, but we wanted the AI to learn on its own, not copy an existing solver.

The Idea

This is the most straightforward approach of all: if you already have a program that can solve the Rubik's Cube (and several exist), just get it to solve millions of cubes, record what it does, and train a neural network to copy it.

This is how most people actually learn to solve the Rubik's Cube — not by figuring it out from scratch, but by watching YouTube tutorials and memorising algorithms. The AI equivalent is called behavioural cloning: watch the expert, copy the expert.

How It Works

1. Scramble a cube randomly
2. Use an existing solver (like Kociemba's algorithm, which can solve any cube in ≤20 moves) to find the solution
3. Record every step: "when the cube looked like this, the expert did that"
4. Repeat millions of times
5. Train a neural network on all these (cube state, correct move) pairs

At solving time, you just show the network a scrambled cube, it predicts the best move, you apply it, and repeat until solved.

The Compounding Error Problem

There's a catch. During training, the network only sees cube states along the expert's solution paths. But when the network solves cubes on its own, it sometimes makes small mistakes — and those mistakes take it to cube states the expert never visited. The network has no idea what to do in these unfamiliar states, so it makes more mistakes, which take it further off course, which leads to even more mistakes…

```
Expert's path:    A → B → C → D → E → SOLVED
Network's path:   A → B → C' → ??? → ??? → LOST
                           ↑
                  Small mistake here snowballs
```

Analogy: It's like learning to drive by watching a perfect driver. You learn what to do when everything is going well. But the moment you make a small mistake (drift slightly left), you're in a situation the perfect driver was never in, and you have no idea how to recover.

Why We Didn't Use It

For our project, imitation learning has a philosophical problem: the AI isn't really learning to solve the cube — it's just copying an existing algorithm. We wanted to demonstrate that AI can figure out the cube through its own experience, which is much more impressive and educational.

There's also a practical issue: the network can never be better than the expert it copies. With ADI, our AI has the potential to discover strategies that no human or algorithm has found before.

Verdict: The fastest path to a working solver, but not the most interesting one. Great for production systems; less great for a project about AI learning.

Approach 6: Hybrid Methods

Planned — our next step is transferring the AI from simulation to the physical robot.

The Idea

In practice, the best results often come from combining multiple approaches. Each technique has strengths that compensate for another's weaknesses. Here are the most interesting combinations:

Sim-to-Real Transfer (Our Next Step)

This is the hybrid approach most relevant to our project. We train the AI entirely in simulation (fast, perfect, free), then adapt it to work on the physical LEGO robot (slow, noisy, real).

```
Phase 1: Train in simulation (what we've done so far)
  - Millions of cubes per hour
  - Perfect information — we know exactly where every sticker is
  - No wear and tear on the robot

Phase 2: Fine-tune on the real robot (our next step)
```

```
- Adapt to real-world messiness:
  - Motors aren't perfectly accurate (turns might be slightly off)
  - Colour sensors sometimes misread stickers
  - The cube might not sit perfectly in the gripper
- Much slower — minutes per cube instead of milliseconds
- But we start from a strong simulation-trained model
```

This is standard practice in robotics — it's called sim-to-real transfer. The simulation does the heavy lifting, and real-world fine-tuning bridges the gap between the virtual and physical worlds.

Other Interesting Combinations

ADI + Better Search: Instead of beam search, you could use Monte Carlo Tree Search (MCTS) — the algorithm that powered AlphaGo's victory over the world Go champion. MCTS is better at exploring deeply along promising paths, while beam search is better at exploring broadly. For the Rubik's Cube, beam search is usually sufficient because solutions are short (≤30 moves), but MCTS might help for the hardest scrambles.

Imitation Learning + RL: Start by copying an expert solver (fast), then use RL to improve beyond the expert (slow but potentially better). This solves RL's biggest problem — the random starting point — by giving the AI a head start.

Verdict: Start simple, add complexity only when needed. For us, ADI alone has been remarkably effective, and the main hybrid element will be sim-to-real transfer when we connect to the physical robot.

Comparison: How the Approaches Stack Up

Here's a summary of all six approaches:

Approach	Difficulty to Build	Training Time	20-Move Solve Rate	Best For
Standard RL	Easy	Hours-Days	~0%	Learning why the problem is hard
Curriculum RL	Easy	Hours	~10%	Moderate difficulty problems
HER	Medium	Hours	Not suitable	Multi-goal problems (not cubes)
ADI	Medium	Hours	25%+ (and improving)	Reliable cube-solving
Imitation Learning	Easy	Minutes	~95% (but copies an expert)	Quick results

Approach	Difficulty to Build	Training Time	20-Move Solve Rate	Best For
Hybrid	Hard	Varies	Potentially the best	Maximum performance

Which Should You Choose?

```
Do you have an existing solver to copy?
├── YES → Just want it to work? → Imitation Learning
│         Want the AI to learn on its own? → ADI
└── NO  → Is there a known goal state?
          ├── YES → ADI (work backwards from the goal)
          └── NO  → Start with Curriculum RL
```

For the Rubik's Cube, the answer is clear: ADI. It's the only approach that reliably solves hard scrambles while genuinely learning from its own experience.

Our Experimental Results

The Story of Five Training Runs

This section tells the story of our project chronologically — each training run, what we hoped would happen, what actually happened, and what we learned. It's a story of failure, iteration, and eventual success.

All experiments ran on an NVIDIA DGX Spark with a GB10 GPU.

Run 1: PPO with Basic Curriculum (January 31, 2026)

What we tried: Standard reinforcement learning (PPO) with a simple curriculum — advance to harder scrambles after 10 consecutive solves. Training took about 2 hours.

What happened: The AI learned to solve 1-6 move scrambles, then got completely stuck at level 7. It couldn't solve a single 10-move or 20-move scramble.

What we learned: The "10 consecutive solves" rule was too easy to fluke — the AI could get lucky without truly mastering a level. Also, the network (394K parameters) was probably too small. But the bigger issue was the sparse reward problem: at level 7, random exploration almost never finds the solution.

Run 2: PPO with Improved Curriculum (February 2, 2026)

What we tried: A 3× bigger network (1.1M parameters), stricter advancement (85% solve rate over 50 attempts), and 3× more training (30M steps, ~7 hours).

What happened: The AI got better at level 7 (54% vs 35%) and could even solve 9% of 10-move scrambles. But it still couldn't break past level 7.

What we learned: This was the crucial moment. We'd tripled the network size and training time, used a much better curriculum rule, and still hit the same wall. This told us the problem wasn't about tuning — it was fundamental to the RL approach. We needed something completely different.

Run 3: ADI v1 (February 3, 2026)

What we tried: A completely different approach — Autodidactic Iteration. Instead of RL, we generated 1 million training examples by scrambling from the solved state, then trained a much larger network (12.6M parameters) using supervised learning. Training took just 20 minutes.

What happened: A complete transformation. The results speak for themselves:

Scramble Depth	Solve Rate
5 moves	100%
10 moves	100%
15 moves	19%
20 moves	4%

100% on 10-move scrambles — up from PPO's 9%. And it trained in 20 minutes instead of 7 hours.

What we learned: ADI works. The switch from RL to supervised learning was the single most important decision in the project. We also discovered that wider beam search dramatically improves results — using beam width 200 instead of 100 pushed the 15-move rate from 19% to 50%. The remaining challenge was 15-20 move scrambles, which suggested we needed more training data.

Run 4: ADI v2 (February 6, 2026)

What we tried: Same network architecture as v1, but with 5× more training data (5M samples instead of 1M) and twice as many epochs. Training took about 3.5 hours.

What happened:

Scramble Depth	v1 (1M samples)	v2 (5M samples)
5 moves	100%	100%
10 moves	100%	99%
15 moves	19%	42%
20 moves	4%	25%

More data clearly helps — the 15-move rate more than doubled, and the 20-move rate went from 4% to 25%.

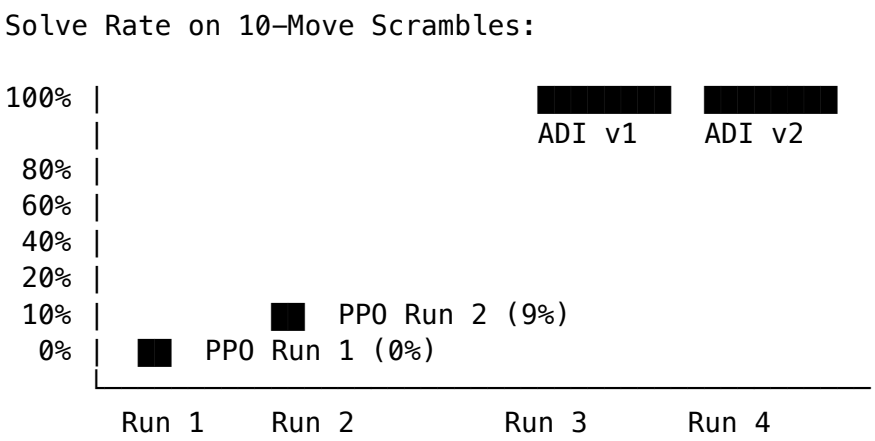
What we learned: The network was running out of capacity. The 20-move results were volatile across training (bouncing between 3% and 25%), suggesting the 12.6M parameter network was at the limit of what it could learn. We needed a bigger, smarter architecture.

Run 5: ADI v3 (February 7, 2026) — In Progress

What we're trying: Our custom-designed network with residual connections (44.7M parameters — 3.5× larger than v1/v2), trained on 20M samples (20× more than v1). This is the architecture described in the ADI section above. Training takes about 14 hours on the DGX Spark.

Early results (halfway through training): Already showing improvement at 15-move depth (58% vs v2's 42%) and 20-move depth (30% vs v2's 25%). The residual connections and extra capacity are paying off.

The Big Picture: How Each Run Compared



The jump from PPO to ADI wasn't a small improvement — it was a complete transformation. ADI addresses the root cause of the problem (sparse rewards) rather than trying to work around it. And within ADI, more data and a bigger network produce consistent, predictable improvements.

What We'd Recommend

If you're building your own AI cube solver, use ADI. Here's why:

- It works: 100% solve rate on 10-move scrambles, and improving on harder ones
- It's fast to train: 20 minutes for v1 (vs 7 hours of PPO that barely worked)
- It scales predictably: More data = better results. Bigger network = better results. No guesswork.
- It's fast enough for a robot: Solutions take 0.04-1.5 seconds, which is fine when the robot takes several seconds per physical move anyway

ADI also works for any puzzle with a known goal state — sliding puzzles, maze navigation, and similar problems. The key requirement is that you can scramble backwards from the goal to generate training data.

The most important lesson from our project: start simple and let failures guide you. We didn't jump straight to ADI — we tried PPO first, learned why it failed, tried curriculum learning, learned why that failed, and only then found ADI. Each failure taught us something essential about the problem.

Further Reading

If you want to dive deeper into any of the topics covered in this guide, here are the key papers and resources:

Our main inspiration: - McAleer et al. (2019) — "Solving the Rubik's Cube with Deep Reinforcement Learning and Search" (Nature Machine Intelligence). This is the DeepCubeA paper that introduced ADI. The approach we based our project on.

The algorithms we tried: - Schulman et al. (2017) — "Proximal Policy Optimization Algorithms". The PPO algorithm we used in Runs 1 and 2. - Mnih et al. (2015) — "Human-level control through deep reinforcement learning" (Nature). The DQN paper — the AI that learned to play Atari games. - Andrychowicz et al. (2017) — "Hindsight Experience Replay". The HER technique we considered but didn't use.

Key background: - He et al. (2015) — "Deep Residual Learning for Image Recognition". The ResNet paper that introduced residual connections — the key innovation in our v3 network. Over 200,000 citations. - Rokicki et al. (2010) — "God's Number is 20" (cube20.org). The proof that any Rubik's Cube can be solved in 20 moves or fewer. - Silver et al. (2017) — "Mastering the game of Go without human knowledge" (Nature). AlphaGo Zero — the AI that beat the world Go champion.

Glossary

A quick reference for all the technical terms used in this guide. Most of these are explained in detail in the main text — this is just for looking things up.

Term	What It Means
Action	Something the AI can do — for us, one of 12 face rotations
ADI	Autodidactic Iteration — our chosen approach. Generates training data by scrambling backwards from solved
Agent	The AI making decisions
Backpropagation	The process of figuring out which weights caused an error, by working backwards through the network
Batch	A group of training examples processed together (ours: 1,024 at a time)
Batch Normalisation	A technique that keeps the numbers flowing through the network in a reasonable range, preventing training instability
Beam Search	A search algorithm that explores many solution paths at once, keeping only the most promising ones
Beam Width	How many paths beam search explores simultaneously (we use 100-500)
Behavioural Cloning	Training an AI to copy an expert's decisions
Catastrophic Forgetting	When learning new things causes the network to forget old things
Curriculum Learning	Training on easy problems first, then gradually harder ones
DeepCubeA	The UC Irvine research system that inspired our approach — achieved 100% solve rate
DGX Spark	Our training hardware — an NVIDIA workstation with a powerful GPU

Term	What It Means
DQN	Deep Q-Network — an RL algorithm that learns how good each action is
Dropout	Randomly turning off neurons during training to prevent memorisation
Environment	The world the AI interacts with — our Rubik's Cube simulator
Epoch	One complete pass through all the training data
God's Number	20 — the maximum moves needed to solve any Rubik's Cube
GPU	Graphics Processing Unit — specialised hardware that does many calculations in parallel, essential for training neural networks
Gradient	The direction to adjust weights to reduce error
Gradient Descent	The core learning algorithm — repeatedly nudge weights downhill to reduce error
HER	Hindsight Experience Replay — pretending failed attempts were actually targeting a different goal
Inference	Using a trained network to make predictions (as opposed to training it)
Layer	A group of neurons that processes data together in the network
Learning Rate	How big the weight adjustments are at each training step
Loss	A number measuring how wrong the network's predictions are (lower = better)
Neuron	A single unit in the network that multiplies inputs by weights and adds them up
Neural Network	A mathematical function made of layers of neurons that can learn patterns from data
One-Hot Encoding	Representing categories (like colours) as vectors of 0s and 1s
Overfitting	When the network memorises training data instead of learning general patterns
Parameter (Weight)	A single adjustable number in the network — our v3 has 44.7 million of them
Policy	The AI's strategy — its rule for deciding what to do
Policy Head	The part of our network that predicts which move to make
PPO	Proximal Policy Optimization — an RL algorithm that learns a strategy directly
ReLU	An activation function: passes positive numbers through, sets negatives to zero
Residual Connection	A shortcut that lets data skip over layers, solving the vanishing gradient problem
Reward	The score the AI gets — +100 for solving, -1 per move
Reward Shaping	Giving partial credit to guide learning (e.g., bonus for correct stickers)
Sim-to-Real	Training in simulation, then adapting to a physical robot
Sparse Reward	When positive feedback is extremely rare — the Rubik's Cube's main challenge for RL
State	What the AI can see — the current cube configuration (324 numbers)
Supervised Learning	Learning from examples with known correct answers

Term	What It Means
Transfer Gap	When skills learned on easy problems don’t fully apply to harder ones
Value Function	The AI’s estimate of how far a cube state is from being solved
Value Head	The part of our network that predicts distance to solved
Vanishing Gradient	When the learning signal fades to nothing in deep networks — solved by residual connections

Our Hardware

Component	Details
Training Computer	NVIDIA DGX Spark
GPU	NVIDIA GB10
Software	Python 3.11, PyTorch, Gymnasium
Robot	LEGO MindCuber (set 51515) with SPIKE Hub

Document created: February 3, 2026 Last updated: February 7, 2026 Project: MindCuber RL — Teaching a LEGO Robot to Solve the Rubik’s Cube with AI